

Market Basket Analysis on the IMDB Top 1000 Movies Dataset

Algorithms for Massive Data – Master in Data Science for Economics
Università degli Studi di Milano

Anastasiia Fokina
48027A

2025/2026

Abstract

This project implements a market basket analysis system on the IMDB Top 1000 Movies dataset. Each movie is treated as a basket and its four listed actors as items. Two algorithms are applied and compared: the A-Priori algorithm and the PCY algorithm (Park, Chen, Yu). The analysis reveals co-occurrence patterns, including well-known cast ensembles from franchises such as the Harry Potter series or Kill Bill.

1 Dataset

The dataset used is the *IMDB Top 1000 Movies and TV Shows* dataset. It contains metadata for 1000 highly rated titles, including title, release year, genre, IMDB rating, and four actor fields: `Star1`, `Star2`, `Star3`, and `Star4`.

For this project, only the four actor columns and the movie title are used. All 1000 rows are considered in the full dataset configuration. A global variable `USE_FULL_DATASET` allows switching between entire dataset and a smaller sample of configurable size, allowing reproducible experiments at different scales.

2 Data Organization

Each row of the dataset is mapped to a *basket*: a set of up to four actors appearing in that movie. Missing values in actor fields are dropped. The resulting list of baskets is then encoded into a binary matrix using the `TransactionEncoder` from the `mlxtend` library, where rows represent movies and columns represent unique actors. A cell contains `True` if the corresponding actor appears in the corresponding movie.

The dataset contains 1000 movies and approximately 3606 unique actors after preprocessing.

3 Preprocessing

The preprocessing pipeline consists of the following steps:

1. Loading the CSV file into a `pandas DataFrame`.
2. Selecting only the four actor columns (`Star1-Star4`).
3. Dropping `NaN` values within each row and removing duplicate actor names within the same basket.
4. Converting each row into a Python list of actor name strings.
5. Encoding the list of baskets into a boolean matrix via `TransactionEncoder`.

No text normalization (e.g. lowercasing) was needed as actor names are consistently formatted in the dataset.

4 Algorithms

4.1 A-Priori

The A-Priori algorithm is a classical method for mining frequent itemsets. It exploits the *anti-monotonicity* property of support: if an itemset is frequent, all of its subsets must also be frequent. Conversely, if an itemset is infrequent, all of its supersets can be pruned without examination.

The algorithm proceeds in passes over the data:

1. Count the support of all individual items and retain those above the minimum support threshold (frequent 1-itemsets).
2. Generate candidate 2-itemsets from pairs of frequent 1-itemsets, count their support, and retain frequent ones.
3. Repeat for increasing itemset sizes until no new frequent itemsets are found.

The algorithm is implemented via the `apriori` function from the `mlxtend` library. The minimum support threshold is defined in absolute terms as a minimum count (default: 2 movies), converted to a relative frequency by dividing by the dataset size. This ensures the threshold adapts correctly when the dataset size changes. The maximum itemset size is set to 4, matching the number of actor fields. Association rules are then extracted using the `association_rules` function with a minimum confidence of 0.5.

4.2 PCY Algorithm

The PCY algorithm (Park, Chen, Yu) improves upon A-Priori by exploiting unused main memory during the first pass. While counting individual items, it also hashes all pairs of items into buckets of a hash table, incrementing the bucket count for each pair. A bucket whose total count falls below the support threshold is called *infrequent*.

On the second pass, a pair $\{i, j\}$ is only counted as a candidate if:

1. Both i and j are frequent items, and
2. The pair $\{i, j\}$ hashes to a *frequent* bucket.

This second condition is the key improvement over A-Priori: pairs that hash to infrequent buckets are pruned without counting, since no frequent pair can hide in a bucket whose total count is below the threshold. The PCY algorithm was implemented from scratch in Python, following Section 6.3 of the course textbook.

5 Scalability

A key requirement of the project is that the solution scales with data size. Several design choices address this:

- The minimum support is defined as an absolute count divided by dataset size, so it scales automatically.
- The `USE_FULL_DATASET` flag and `SAMPLE_SIZE` variable allow running experiments on subsets of any size.
- The A-Priori algorithm reduces the search space via candidate pruning based on anti-monotonicity.
- The PCY algorithm further reduces the number of candidate pairs by filtering via a hash table, making it more memory-efficient on larger datasets.

A scalability experiment was conducted by running A-Priori on samples of increasing size (100, 200, 300, 500, 750, and 1000 movies) and measuring execution time. Results show that execution time grows gradually and remains well under one second for the full dataset.

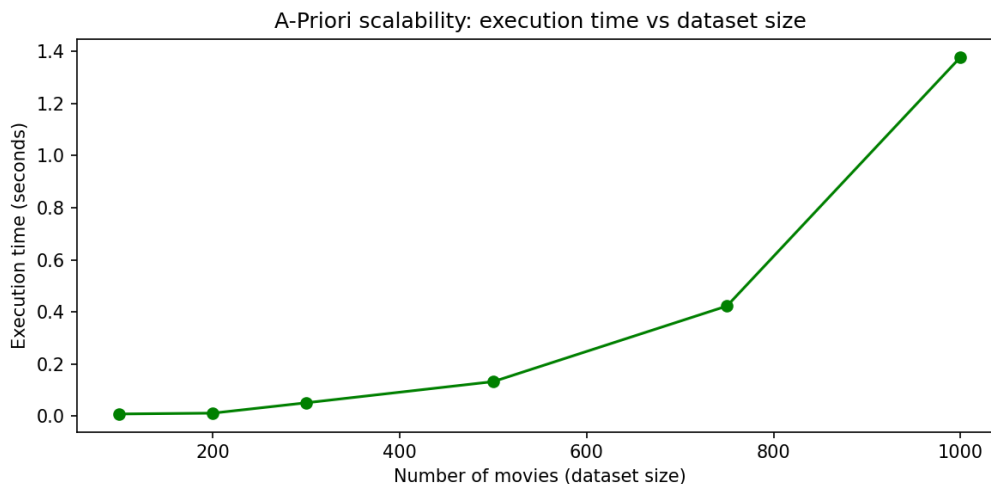


Figure 1: Execution time of A-Priori as a function of dataset size.

6 Experiments

All experiments were run on Google Colab using Python 3.12. The full dataset of 1000 movies was used with a minimum support count of 2 (i.e. actor combinations appearing in at least 2 movies).

6.1 Frequent Itemsets (A-Priori)

The A-Priori algorithm found a total of 791 frequent itemsets distributed as follows:

Itemset size	Count
1	641
2	121
3	25
4	4
Total	791

Table 1: Number of frequent itemsets by size (A-Priori).

6.2 Association Rules

A total of 337 association rules were extracted with minimum confidence 0.5. The top rules by lift involve actors from the same film series, such as the Kill Bill cast (Uma Thurman, David Carradine, Daryl Hannah, Michael Madsen) and classic horror films (Boris Karloff, Colin Clive). These rules achieve a lift of 500, indicating a very strong co-occurrence well beyond what would be expected by chance.

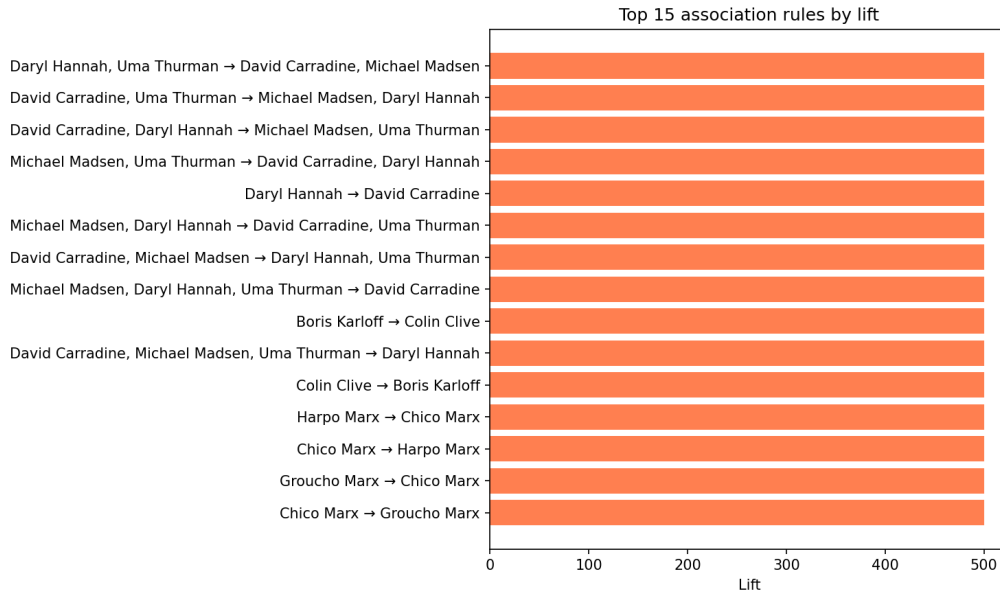


Figure 2: Top 15 association rules ranked by lift.

6.3 PCY Results

The PCY algorithm was run with the same minimum support of 2. On the first pass, it identified 641 frequent items and 991 out of 1000 frequent buckets. On the second pass, only 1671 candidate pairs were checked, yielding exactly the same 121 frequent pairs as A-Priori, confirming the correctness of the implementation.

The top 10 frequent actor pairs discovered by PCY are shown in Table 2, reflecting well-known film franchises.

Actor Pair	Movies together
Daniel Radcliffe, Rupert Grint	6
Daniel Radcliffe, Emma Watson	5
Emma Watson, Rupert Grint	5
Joe Pesci, Robert De Niro	4
Tim Allen, Tom Hanks	4
Al Pacino, Diane Keaton	3
Christian Bale, Michael Caine	3
Al Pacino, Robert De Niro	3
Elijah Wood, Ian McKellen	3
Elijah Wood, Orlando Bloom	3

Table 2: Top 10 frequent actor pairs found by PCY.

6.4 A-Priori vs PCY Comparison

Table 3 summarizes the performance comparison between the two algorithms on the full dataset.

Metric	A-Priori	PCY
Frequent pairs found	121	121
Execution time	0.9551s	0.0073s
Candidate pairs checked	all frequent item pairs	1671
Results agree	Yes	

Table 3: Comparison between A-Priori and PCY algorithms.

PCY is approximately 130 times faster than A-Priori on this dataset. The speedup comes from the hash table filter: pairs that hash to infrequent buckets are skipped entirely on the second pass, dramatically reducing the number of candidates that need to be counted.

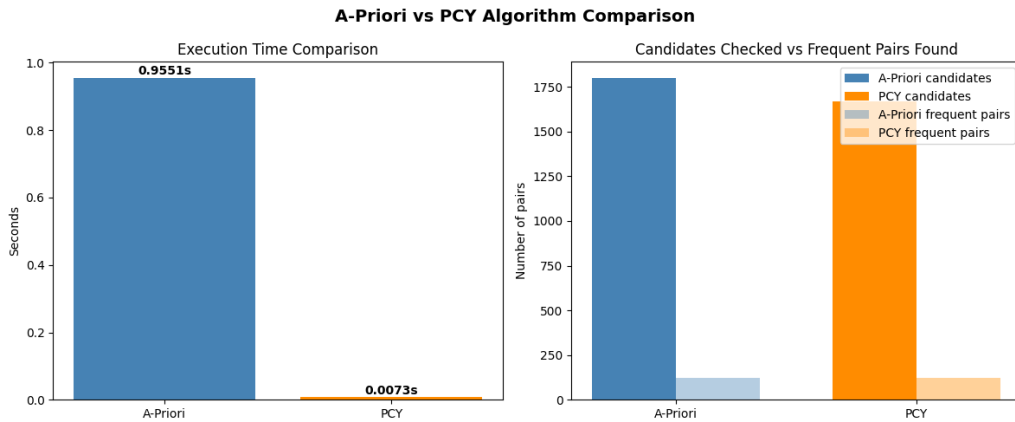


Figure 3: Visual comparison between A-Priori and PCY algorithms.

7 Conclusion

The results confirm that market basket analysis is an effective tool for identifying recurring actor ensembles in film. The strongest associations correspond to actors who appeared together in multiple films of the same franchise or director, such as the Harry Potter cast (Daniel Radcliffe, Emma Watson, Rupert Grint appearing together in 5–6 movies) and the Godfather cast (Al Pacino, Robert De Niro, Diane Keaton).

The comparison between A-Priori and PCY demonstrates the practical benefit of hash-based filtering: PCY checks far fewer candidate pairs while producing identical results. On larger datasets, this difference would be even more pronounced, as the number of candidate pairs grows quadratically with the number of frequent items.

A limitation of this dataset is its small size: with only 1000 movies, most actor pairs appear together in just one film, making low support thresholds necessary. On a larger dataset (e.g. the full IMDB database), more robust and generalizable patterns would emerge.

Declaration

I declare that this material, which I now submit for assessment, is entirely my own work and has not been taken from the work of others, save and to the extent that such work has been cited and acknowledged within the text of my work, and including any code produced using generative AI systems. I understand that plagiarism, collusion, and copying are grave and serious offences in the university and accept the penalties that would be imposed should I engage in plagiarism, collusion or copying. This assignment, or any part of it, has not been previously submitted by me or any other person for assessment on this or any other course of study.